

Relazione Progetto: Simulatore Blackjack Object-Oriented

Tommaso Pierelli

Matricola: 327883

Università degli Studi di Urbino - Carlo Bo

Anno Accademico 2025/2026

Indice

1	ANALISI	2
1.1	REQUISITI	6
1.2	MODELLO DEL DOMINIO	7
2	DESIGN	9
2.1	Architettura	9
2.1.1	Model	9
2.1.2	View	10
2.1.3	Controller	11
2.2	Design dettagliato	12
2.2.1	Model nel dettaglio	12
2.2.2	View nel dettaglio	14
2.2.3	Controller nel dettaglio	18
3	SVILUPPO	21
3.1	Testing automatizzato	21
3.2	Metodologia di lavoro	22
3.2.1	Version Control	22
3.3	Note di sviluppo	22
3.3.1	Feature avanzate	22
3.3.2	Algoritmi non forniti da libreria	22
4	TUTORIAL APP	24
4.1	Scelta della modalità	24
4.2	Menù di inizializzazione	25
4.3	Menù per le bet	26
4.4	Tavolo da gioco	27
4.4.1	Tavolo iniziale	27
4.4.2	Mano sconfitta	28
4.4.3	Esito side bet	28
4.4.4	Mano con split	29

Capitolo 1

ANALISI

L'obiettivo del progetto è quello di realizzare un simulatore single-player del popolare gioco da casinò "Blackjack".

Regole di base

Il Blackjack è un gioco di carte in cui ogni giocatore gioca una partita 1vs1 con il banco. Il mazzo utilizzato è il mazzo di carte "a semi francesi" composto da 52 carte, jolly esclusi. Per ogni seme (cuori, quadri, fiori e picche) il mazzo contiene 13 carte composte da:

- un asso(A)
- 9 carte dal 2 al 10(2, 3, 4, ..., 10)
- un "Jack"(J)
- una "Donna/Regina"(Q)
- un "Re"(K).

Nelle regole del Blackjack le carte assumono valori diversi rispetto ai loro valori in scala. Le 9 carte dal 2 al 10 mantengono il loro valore classico, il J, la Q e il K valgono tutti e tre 10 (a differenza di 11, 12 e 13 come si potrebbe pensare). L'unica carta a poter assumere due valori diversi è l'asso, in particolare assume valore 11 qualora non si fosse ancora raggiunto un punteggio della mano maggiore di 21, assume valore 1 se invece il conteggio delle carte, assumendo l'asso a valore 11, dovesse far sballare il punteggio portandolo oltre il 21. La partita inizia con più mazzi di default mescolati casualmente, comuni al banco e al giocatore. All'inizio di una mano, dopo che il player ha scelto la quantità di denaro da puntare, banco e giocatore ottengono due carte a testa, il giocatore le ha entrambe scoperte, il banco soltanto una, l'altra rimarrà coperta fino alla conclusione della giocata del giocatore. Una volta distribuite le carte, il giocatore avrà la possibilità di fare quattro mosse:

- Hit - ricevi una carta.
- Stand - chiudi le tue giocate e resta con le carte attuali, si passa quindi la palla al banco.
- Double Down - o raddoppio, permette di ricevere una sola carta aggiuntiva raddoppiando la giocata iniziale. Il raddoppio esclude la possibilità di ricevere ulteriori carte a seguito dell'ultima ottenuta. Può essere eseguito solo come prima mossa (con due carte in mano). Valido anche se eseguito dopo uno split.

- Split - giocata eseguibile solo con due carte in mano che abbiano lo stesso valore nominale, divide la mano attuale in due mani separate, permettendo di giocare due partite a sé. Anche in questo caso il costo è raddoppiare la puntata. Alcuni casinò applicano regole diverse, come ad esempio un maggior numero di split per lo stesso turno e un trattamento più restrittivo qualora venisse eseguito su una coppia d'assi (per evitare un eccessivo vantaggio al giocatore, in gran parte dei casinò viene assegnata una sola carta per mano a seguito di uno split eseguito su due assi, senza possibilità di ricevere ulteriori carte). Per semplificare la gestione delle mani il simulatore permetterà di eseguire un solo split come prima mossa e, indipendentemente dalle carte divise, permetterà di giocare le mani separate come fossero mani normali ma senza la possibilità di un secondo o un terzo split. A seguito del primo split sarà possibile per entrambe le mani effettuare Hit, Stand o Double Down, ma non un secondo split.

L'obiettivo finale del giocatore è quello di avvicinarsi il più possibile al punteggio di 21 con le sole carte che ha in mano, cercando di ottenere un punteggio più alto di quello del banco. Il giocatore, a meno che non scelga di pescare una carta raddoppiando, è libero di pescare senza limiti, fino al raggiungimento di un punteggio complessivo dal 21 in su. Qualora la somma dei punteggi delle singole carte fosse maggiore di 21 la mano si direbbe sballata, il che porterebbe a una sconfitta automatica. Il banco, a differenza del giocatore, possiede un vincolo, è infatti costretto a pescare carte fino al raggiungimento del punteggio minimo di 17. Fino a che la somma è minore o uguale a 16, il banco continua a chiamare carta. In caso di situazione di 17 "morbido" (quando il banco raggiunge 17 con un asso a valore 11) il banco continua a pescare e non si ferma sul 17, è l'unico caso in cui ha la possibilità di sviare la regola principale. Il banco non può raddoppiare la posta e nemmeno splittare le carte, in quanto solo il giocatore effettua la puntata prima di iniziare la mano. Una volta concluse le giocate del giocatore e del banco, si determina la vittoria secondo questi criteri:

- Se il giocatore ottiene un punteggio superiore al 21, si dice che abbia sballato (bust) e perde automaticamente la puntata.
- Se il giocatore ottiene un punteggio pari a quello del banco c'è un pareggio (push). Al giocatore viene restituita la puntata.
- Se il giocatore ottiene un punteggio maggiore di quello del banco vince la mano. Una vittoria di questo tipo paga 1 a 1.
- Se banco o giocatore ottengono un 21 con sole due carte si verifica il "Blackjack" che dà nome al gioco. In questo caso anche a parità di punteggio (21 pari) ha la meglio chi lo ha ottenuto attraverso il Blackjack. Se si verificano due Blackjack c'è un pareggio. Una vittoria di questo tipo paga 3 a 2. Quando un giocatore ottiene un Blackjack la mano si considera automaticamente conclusa senza dare alcuna possibilità di ulteriore giocata, anche in caso di Blackjack ottenuto dopo uno split.

Side bet: Perfect Pairs

Il simulatore avrà la possibilità di gestire una side bet oltre al normale flusso di gioco della mano. Una side bet è una puntata addizionale che il giocatore sceglie di compiere prima che vengano consegnate le carte. A differenza della puntata principale è facoltativa ed è solo un'aggiunta alla giocata. In particolare verrà gestita la side bet chiamata "Perfect

Pairs” (coppia perfetta). Si tratta di una side bet legata alle due carte che il giocatore ottiene appena iniziata la mano. I casi di vincita possibili sono tre:

- Se il giocatore ottiene due carte con lo stesso valore e lo stesso seme (ad esempio due 8 di cuori) la vincità sarà di 25:1.
- Se il giocatore ottiene due carte dello stesso valore, con semi diversi, ma dello stesso ”colore” (ad esempio un 8 di fiori e un 8 di picche, entrambe carte nere) la vincità sarà di 12:1.
- Se il giocatore ottiene due carte dello stesso valore, con semi diversi e di colore diverso (ad esempio un 8 di fiori e un 8 di cuori) la vincità sarà di 6:1.
- Se il giocatore ottiene due carte di valore diverso la side bet sarà persa, la puntata del giocatore andrà perduta e si proseguirà con la puntata principale.

Và tenuto conto del fatto che i valori delle carte per il calcolo della side bet è quello originale delle carte e non quello secondo le regole del Blackjack, ad esempio una Q e un K non costituiscono una coppia perfetta nonostante nelle regole del Blackjack abbiano entrambe valore 10.

Assicurazione

Il simulatore offrirà al giocatore la possibilità di ”assicurazione” nel caso in cui la carta scoperta del banco sia un asso (con appositi tasti per scegliere se assicurarsi oppure no). Se il giocatore sceglie di assicurarsi, dovrà obbligatoriamente puntare la metà della puntata offerta nella mano principale. A quel punto si aprono due scenari:

- Se il banco ha Blackjack l’assicurazione paga 2:1. Quindi il giocatore recupera il valore della sua puntata principale (che perde, perché il banco ha Blackjack) tramite la vincita dell’assicurazione (di fatto va in pareggio sul totale). Se anche il giocatore ha Blackjack, la mano va in pareggio, si recupera la puntata principale, e si chiude in positivo con la vincita dell’assicurazione
- Se il banco non ha Blackjack i soldi di assicurazione scommessi si perdono e si prosegue giocando la mano principale.

In generale va specificato che l’assicurazione è una giocata che a lungo termine penalizza il giocatore, in quanto la probabilità di un Blackjack al banco è minore di ogni altra combinazione, inoltre l’unico caso in cui si esce in positivo è il doppio Blackjack (del giocatore e del banco), una combinazione a bassissime probabilità.

Sistema di timing

Il simulatore offrirà un sistema di timing, che costringerà il giocatore durante la partita a fare la sua giocata entro un certo limite di tempo (20 secondi) per simulare al meglio le politiche dei veri casinò. Qualora il giocatore dovesse terminare il tempo a sua disposizione senza aver compiuto una mossa valida il sistema lo costringerà ad uno ”stand” e la mano verrà calcolata con le carte a sua disposizione in quel momento. Il sistema di timing dei 20 secondi vale anche per la scelta dell’assicurazione. In caso di mancata scelta, allo scadere del tempo, il sistema rifiuta automaticamente l’assicurazione proseguendo normalmente la partita.

Menù e configurazioni

Il software comprende un menù all'avvio che permette all'utente di scrivere il proprio nome o nickname e il saldo con il quale vuole entrare in partita. Inoltre saranno presenti delle sezioni per la personalizzazione delle texture e la gestione di eventuali regole del banco configurabili. Le regole configurabili sono:

- La modifica del comportamento del banco in una situazione di 17 "morbido", costringendolo a fermarsi necessariamente al raggiungimento di un punteggio pari a 17
- La scelta del numero di mazzi, con un numero minimo concesso di 2 e un numero massimo di 8, come in gran parte dei casinò reali. Se ci fosse la possibilità di giocare con un solo mazzo da 52 carte non sarebbe in alcun modo possibile la vincita della "Perfect Pair" che paga 25 a 1. In un solo mazzo non esistono carte esattamente uguali, questo fa perdere valore alla side bet e la rende molto sconveniente al giocatore.

1.1 REQUISITI

Requisiti funzionali

- Il simulatore deve permettere di gestire una classica partita di Blackjack seguendone le regole.
- Deve essere messo a disposizione dell'utente un menù chiaro per inserire il proprio nome utente e il saldo con cui entrare in partita ed effettuare eventuali modifiche allo stile di gioco.
- Devono essere implementate le quattro mosse possibili per il giocatore con annessi bottoni specifici per Hit, Stand, Double Down e Split. Inoltre deve essere messo a disposizione un bottone per acquistare l'assicurazione quando possibile.
- Deve essere gestito un sistema di conto, con cui il giocatore può entrare in partita.
- Devono essere implementate le funzionalità e la gestione della side bet "Perfect Pairs".
- Il simulatore deve implementare un sistema di timing per le mosse del giocatore, in modo che non trascorra troppo tempo a pensare, simulando un vero casinò.
- Il simulatore deve implementare un sistema di assicurazione per il giocatore nel caso in cui la prima carta del banco fosse un asso.
- Deve essere implementata la possibilità dal menù di configurare le regole del gioco modificabili. In particolare il comportamento del banco sul 17 "morbido" e il numero di mazzi con cui giocare, da un minimo di 2 a un massimo di 8.
- Deve essere implementata la scelta di temi per le carte e per il tavolo da gioco.
- Deve essere implementato un sistema di suoni per le principali attività del simulatore.

Requisiti non funzionali

- Il sistema deve garantire la stabilità del software intercettando eventuali operazioni illegali e gestendo eventuali errori evitando che il sistema vada in crash.
- Il sistema deve essere progettato con un'architettura modulare, facilmente scalabile qualora si volessero aggiungere ulteriori giochi in stile casinò permettendo di riutilizzare le stesse componenti.
- La gestione delle operazioni deve essere rapida per rendere fluida l'esperienza di gioco.
- Il sistema deve rendere prontamente visibili lo stato della mano e i bottoni per le operazioni a disposizione del giocatore in modo che il timer non risulti un ostacolo frustrante ma una pressione da vero casinò.
- Il sistema deve garantire equità nel mescolamento dei mazzi e nella consegna delle carte per garantire la casualità delle giocate, rendendole più eque e meno prevedibili.

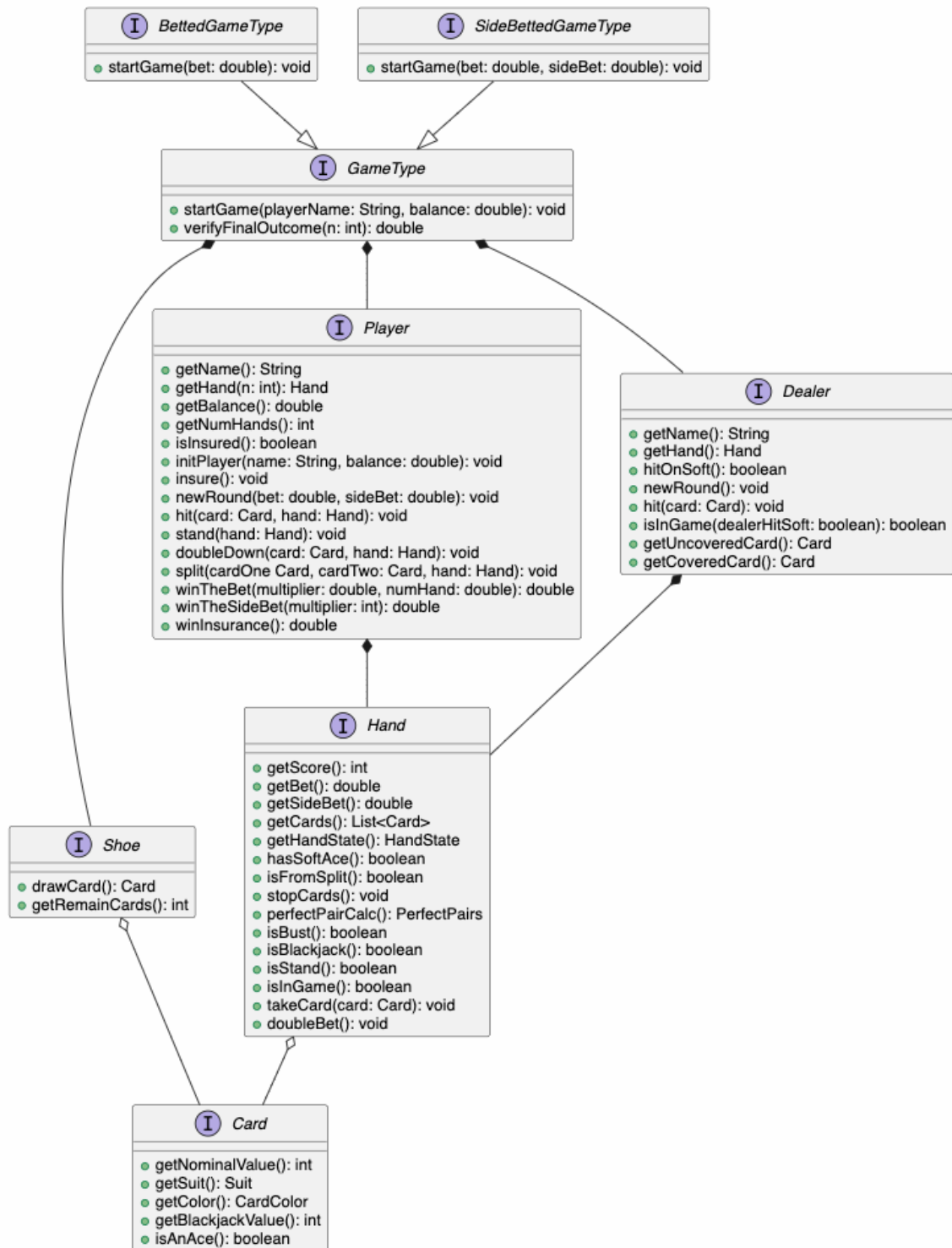
1.2 MODELLO DEL DOMINIO

Il simulatore di Blackjack dovrà poter gestire una normale partita di Blackjack secondo le regole citate nelle precedenti sezioni.

Entità

Le entità in gioco nel sistema sono le seguenti:

- **Card:** è l'entità che rappresenta la carta da gioco, costituita da un seme (cuori, quadri, fiori, picche), da un valore nominale e da un valore utilizzato per il conteggio dei punti.
- **Shoe:** è l'entità che rappresenta l'insieme di più mazzi singoli da 52 carte, dalla quale giocatore e banco pescano le singole carte durante la partita.
- **Hand:** è la mano del giocatore o del banco in cui sono presenti le carte attualmente in gioco. Memorizza il punteggio corrente, il suo stato (Stand, Bust, Blackjack o attiva), la puntata ad essa legata e un'informazione che indica se la mano deriva da uno Split.
- **Player:** è l'entità che rappresenta il giocatore. Il giocatore ha un nome utente, un saldo, una Hand e, in caso di Split, una seconda Hand e uno stato per l'assicurazione.
- **Dealer:** è l'entità che rappresenta il banco. Come il Player possiede una Hand. Possiede solo un nome simbolico e non ha né un conto né uno stato di assicurazione.
- **GameType:** è l'entità che modella la tipologia di gioco e contiene le informazioni comuni ad ogni possibile gioco del sistema.
- **BettedGameType:** è l'entità che estende GameType e che contiene metodi comuni ad ogni gioco da casinò che richiede solo una puntata principale.
- **SideBettedGameType:** è l'entità che estende GameType e che contiene metodi comuni ad ogni gioco da casinò che richiede sia una puntata principale, sia una puntata laterale, è il caso del Blackjack.
- **Blackjack:** concretizza SideBettedGameType. Rappresenta il sistema di gioco nel suo complesso e coordina l'interazione tra tutte le entità del dominio, gestendo il flusso della partita e l'applicazione delle regole.



Capitolo 2

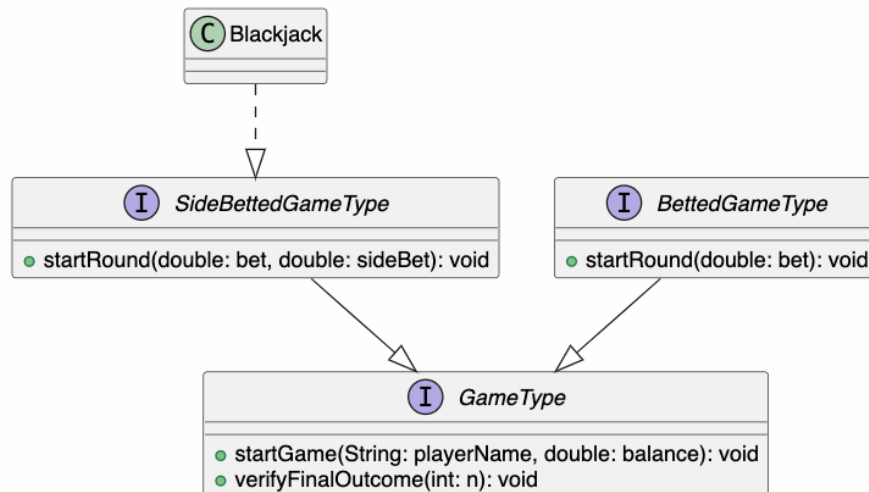
DESIGN

2.1 Architettura

Il modello che si è scelto di seguire per la modellazione dell'applicazione è quello MVC (Model-View-Controller) poiché considerato il più adatto per la gestione del flusso di gioco di una tipica partita di Blackjack. A prova della corretta implementazione dell'architettura, che prevede una precisa separazione dei compiti e delle responsabilità tra le tre componenti, è stata pensata anche una semplice versione CLI(Command Line) dell'applicazione, oltre a quella GUI. L'applicazione CLI, nonostante la semplicità del design, mantiene al suo interno la corretta logica per quanto riguarda regole e stati di una normale partita di Blackjack.

2.1.1 Model

Il model è la parte del sistema che si occupa di tenere traccia ed eventualmente modificare gli stati di una partita. Il Blackjack è un gioco che nel pieno delle sue funzionalità tende a portarsi dietro diversi stati di gioco per mantenere tutte le informazioni e combinarle, permettendo la corretta applicazione delle regole. In un'architettura MVC il model comunica solo con il controller, che riceve da lui dati, e glieli passa per aggiornare lo stato in base alle scelte del giocatore. Per implementare queste funzionalità, la scelta è ricaduta sull'implementazione di interfacce che possano portare al loro interno metodi implementabili e comuni a quasi tutti i giochi in stile casinò pensando a possibili future implementazioni e features. In particolare, è stata pensata un'interfaccia molto generica `GameType`, che viene estesa da due diverse interfacce `BettedGameType` e `SideBettedGameType`, per distinguere i giochi che richiedono anche sidebets (è il caso del Blackjack) e giochi in cui si punta solo direttamente. Coerentemente con questa filosofia di progettazione, ogni classe implementativa del model (`CardImpl`, `HandImpl`, `PlayerImpl`, `DealerImpl`, `ShoeImpl`) possiede una propria interfaccia (`Card`, `Hand`, `Player`, `Dealer`, `Shoe`). Questa scelta permette di esporre al controller e, indirettamente, alla view solo il contratto pubblico necessario, nascondendo i dettagli implementativi e agevolando l'introduzione di implementazioni alternative, qualora necessario. Inoltre, separare interfaccia e implementazione facilita eventuali futuri cambiamenti nella logica interna delle classi senza impattare le componenti che le utilizzano.



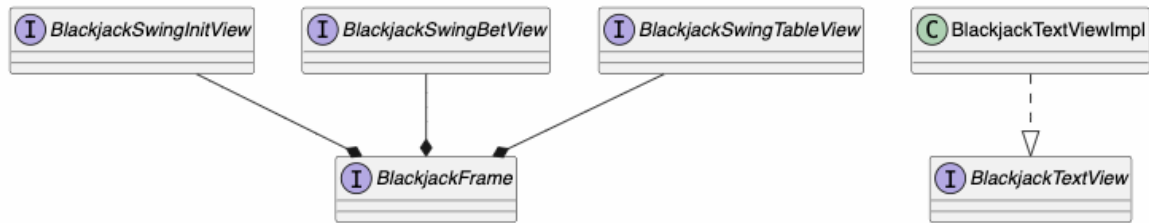
2.1.2 View

La view è la parte dell'architettura che si occupa di mostrare all'utente le modifiche delle schermate di gioco, in base alle informazioni ricevute dal controller, che a sua volta comunica con il model. Di fondamentale importanza è la corretta separazione di model e view, che non devono comunicare in alcun modo. Proprio questa attenzione è stata oggetto di diverse ore di implementazione della view, che riceve dal controller solo dati atomici direttamente da mostrare, lasciando la parte di recupero dei dati ed eventuale calcolo al controller. La view è stata implementata sfruttando la libreria Swing di Java. Si tratta di una libreria abbastanza semplice da usare una volta che si entra nel meccanismo e si capisce il modo in cui comunicano i vari elementi in un frame. L'applicazione prevede tre diverse schermate. Si è scelto per questo di implementare ognuna di queste con una classe dedicata, dotata della propria interfaccia. Questa scelta deriva dal voler separare in maniera chiara e ordinata i compiti, sia per facilitare navigazione nel codice ed eventuali correzioni e nuove implementazioni, sia per evitare la scrittura di una "God Class", ovvero una classe con eccessive responsabilità. In particolare le interfacce per le tre schermate della versione GUI sono:

- **BlackjackSwingInit**: per il menu di inizializzazione del gioco.
- **BlackjackSwingBet**: per la schermata di inizio round in cui si raccolgono le bet.
- **BlackjackSwingTable**: per la schermata che mostra il tavolo di gioco, le carte, i punteggi e i vari esiti.

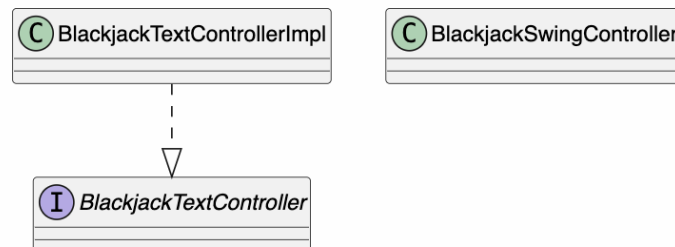
Tutte le tre interfacce sono contenute in quella più generale che gestisce il frame generale dell'applicazione (**BlackjackFrame**) e gioca il ruolo di coordinatore per il controller che mantiene al suo interno solo un'istanza di questa classe. Anche la view per la versione CLI dell'applicazione possiede una sua interfaccia personale. Pur presentando alcuni metodi comuni o simili, le due versioni dell'applicazione non si prestano ad interfacce comuni essendo i metodi implementati con logiche estremamente differenti e ricevendo parametri diversi. Da queste considerazioni la scelta di separare le view della versione GUI e la

versione CLI.



2.1.3 Controller

Il controller in un'architettura MVC funge da coordinatore tra model e view. Mantiene al suo interno un'istanza di entrambe e utilizza metodi getter del model e metodi della view per mostrare dati aggiornati o, viceversa, aggiornare dati in base alle scelte dell'utente raccolte dalla view. L'applicazione offre due controller differenti, uno per la versione CLI e uno per la versione GUI. Le due versioni del gioco presentano flussi di gioco radicalmente differenti. Questo porta i due controller a non avere quasi nessun punto in comune e quindi a non prestarsi a interfacce comuni. Il controller della versione CLI presenta un'interfaccia per l'unico metodo pubblico posseduto che gestisce il flusso di gioco. La versione GUI non avendo un flusso sequenziale ma a eventi, non presenta metodi pubblici da chiamare per far partire il gioco; per questo non è stata implementata un'interfaccia specifica.



2.2 Design dettagliato

2.2.1 Model nel dettaglio

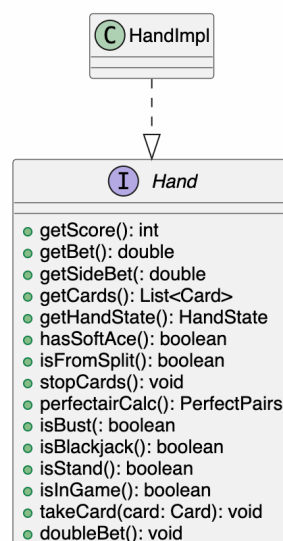
Il model dell'applicazione si occupa della gestione di tutti gli stati che una partita si porta dietro. Inizialmente sono state implementate le classi che gestiscono gli elementi più elementari del gioco, come le carte e il mazzo. Le classi più ricche e degne di nota sono la classe **HandImpl** e la classe **PlayerImpl**, classi con il compito di mantenere un gran numero di stati per ogni round.

HandImpl

HandImpl è la classe che implementa l'interfaccia Hand. Modella il concetto di mano di gioco a cui sono legate tutte le sue informazioni. I diversi stati della mano in base alle regole che sono state implementate sono:

- La presenza di un asso da valore 11, per il calcolo del punteggio secondo le regole del gioco e secondo le impostazioni del dealer.
- Lo stato di una mano in base alle carte ottenute.
- La derivazione della mano, se da uno split oppure no, in modo da poter gestire eccezioni.
- Il tipo di coppia ottenuta, in modo da poter consegnare o decurtare i soldi puntati dal giocatore nel modo giusto.

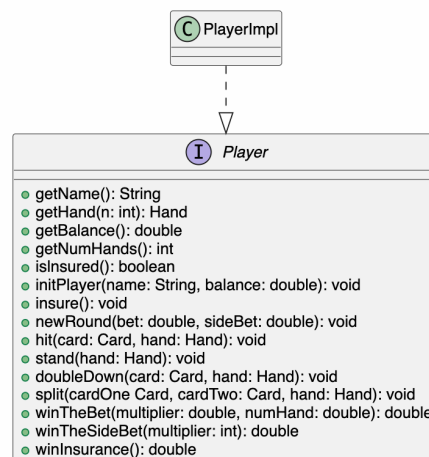
In questa classe è presente un metodo che calcola il punteggio e restituisce il valore per ulteriori calcoli. Inoltre, ad ogni carta pescata, viene chiamato un metodo che aggiorna lo stato della mano. La stessa classe lancia eccezioni nei casi in cui si provi a compiere operazioni non consentite, come uno split ad una mano già derivante da uno split.



PlayerImpl

PlayerImpl è la classe che implementa l'interfaccia Player e che gestisce la figura del giocatore. Il giocatore possiede un nome e un conto, che vengono inizializzati tramite un metodo. Poi possiede una lista di mani (il che prevede una possibile seconda mano dopo uno split) e uno stato che indica se si è assicurato oppure no. I metodi messi a disposizione sono quelli tipici per le 4 mosse del Blackjack. Di particolare attenzione è il metodo che permette lo split. All'interno del metodo vengono create due mani diverse; la prima carta della mano originale viene posta nella prima mano derivante dallo split, mentre la seconda carta va nella seconda mano. Infine, viene fatta pescare una carta a ognuna delle due mani. La presenza di due possibili mani comporta la necessità di controllare la presenza di entrambe ed eventualmente la loro gestione. La classe è dotata di un metodo che permette di assicurarsi, togliendo mezza bet dal conto generale e assegnando valore true alla variabile di stato nominata in precedenza. Sono inoltre presenti metodi per la vittoria delle varie scommesse, in particolare tre metodi separati; uno per l'assicurazione, uno per la side bet e uno per la bet principale. Questi metodi non compiono alcun controllo, si preoccupano solo di consegnare al giocatore il denaro corrispondente alla vincita, restituendone il valore. La side bet e la main bet sono gestite attraverso moltiplicatori come parametro, in modo da passare al metodo il giusto valore con cui moltiplicare il valore della scommessa.

La classe regina che coordina tutte le altre e gestisce le principali attività di gioco e comunica con il controller è la classe Blackjack.



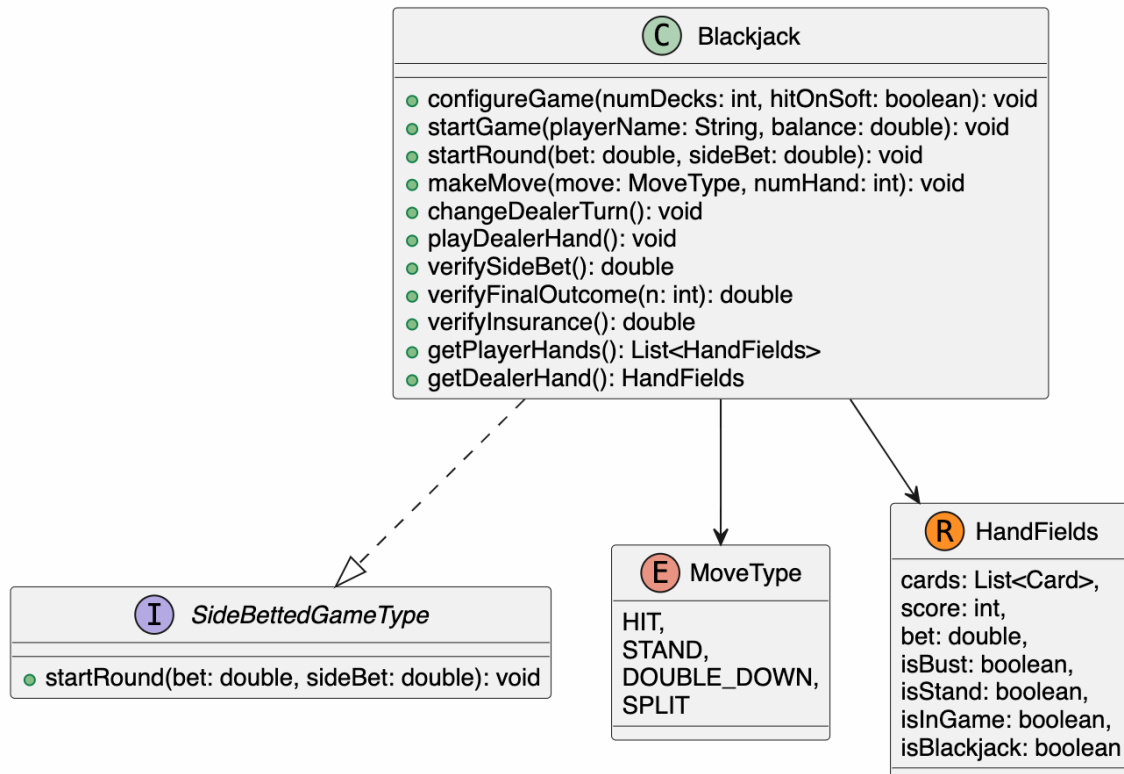
Blackjack

Classe pensata come classe madre del model, gestisce le principali azioni di gestione e aggiorna gli stati più interni del model, lasciandoli il più nascosti possibile al controller. Per questo motivo la classe è ricca di getter utili per il controller, tuttavia sono metodi che evitano lunghe catene di get, per mantenere l'incapsulamento e tenere separate le responsabilità. Blackjack offre metodi per l'inizializzazione di una partita e di un round ricevendo dati dal controller, a sua volta ricevuti dalla view. In questa classe vengono svolti i principali calcoli per modificare gli stati di gioco, in particolare:

- Vengono gestite le mosse e quindi poi richiamati i giusti metodi.
- Vengono gestiti i turni e si calcola quando è il momento di cambiare da player turn a dealer turn.

- Viene effettuato il calcolo dell'esito delle scommesse possibili ed eventualmente richiamati i metodi per consegnare le vincite.

La classe possiede anche un campo configurations, si tratta di un'istanza della classe ConfigurationImpl. Questo campo viene inizializzato nel metodo di configurazione della partita con i dati ricevuti dal controller. Contiene le informazioni sul tipo di dealer scelto dal giocatore e sul numero di mazzi con cui si giocherà.



2.2.2 View nel dettaglio

Come spiegato nelle sezioni precedenti, la view è stata spezzata in classi differenti in base alle funzionalità messe a disposizione, ognuna con la propria interfaccia. Entrambe le versioni dell'app sviluppate hanno elementi comuni, ma la loro implementazione differisce profondamente.

View CLI

Nella versione CLI, tutte le letture dei dati inseriti dall'utente sono lette direttamente da uno scanner che valuta la riga direttamente sottostante alla domanda proposta; ne viene salvato il contenuto e poi viene convertito nel tipo di dato richiesto per poter passare il dato al controller. In questa versione, ad esempio, la gestione dell'input di bet e side bet avviene separatamente, effettuando lo scan in modo separato così come il controllo di eventuali input non accettabili.

Stampa del tavolo

La stampa del tavolo viene eseguita attraverso l'uso di stampe sul terminale che mostrano i principali dati di una partita necessari al giocatore. In particolare, è stata separata in tre diversi metodi: uno che mostra il tavolo iniziale, uno che mostra il tavolo mano a mano che il giocatore compie le sue mosse e uno che mostra il tavolo nella sua versione finale, con la mano giocata dal dealer. Gli ultimi due prevedono anche la gestione della possibile presenza di una seconda mano attraverso dei cicli.

Gestione degli esiti

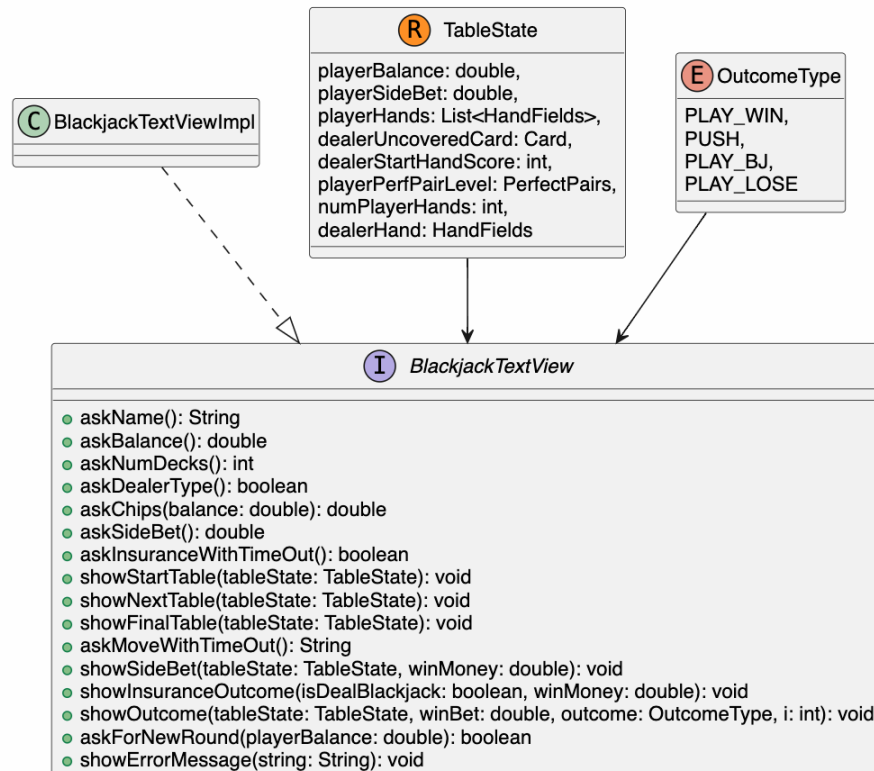
Anche la gestione degli esiti è controllata da metodi separati, uno per ogni bet possibile: assicurazione, side bet e main bet. Qui non vengono eseguiti calcoli, ma viene semplicemente mostrato l'esito in base allo stato calcolato precedentemente nel model e comunicato alla view dal controller, come prevede l'architettura MVC.

Gestione del timer

La gestione del timer è stata una delle principali sfide da affrontare per entrambe le versioni. Nella versione CLI, vista la modalità di lettura dei dati attraverso uno scanner, non è possibile nativamente bloccare la lettura senza che il buffer di input venga corrotto per le letture successive. Per questo, è stato implementato un ciclo di lettura del buffer di input non bloccante che, ad ogni iterazione, controlla che ci siano dati da leggere e, in caso affermativo, li salva; questo avviene fino alla lettura di un newline. A seguito del newline, viene valutata la risposta e si decide se l'input è valido oppure no. Una volta terminato il tempo prestabilito, se l'utente non ha digitato nulla di valido, viene forzata la scelta. La stessa struttura viene utilizzata sia per il metodo che chiede l'assicurazione, sia per il metodo che chiede la mossa da eseguire.

Record

In questa versione, visti i numerosi dati da passare ai metodi che mostrano il tavolo di gioco, è stato pensato l'utilizzo di Record. I Record mantengono al loro interno lo stato di alcuni dati di giocatore e mani quando inizializzati, in maniera immutabile. In questo modo la view può accedervi ma senza possibilità di conoscere parti di metodi e classi del model.



View GUI

La versione Swing della view è stata suddivisa principalmente in quattro classi, tre di queste mostrano esclusivamente i dati relativi alla schermata che implementano, una è la classe che gestisce le altre tre. In generale, in questa versione, è stato scelto un pattern **listener-observer**, il quale prevede che la view, ad ogni interazione dell'utente con l'interfaccia grafica, mandi un segnale al controller che svolge il ruolo di ascoltatore, e riceve aggiornamenti dalla view. In tutte e tre le classi relative alle schermate, il costruttore crea la base principale della struttura grafica sfruttando eventualmente metodi privati, e mette a disposizione dell'utente spazi con cui interagire. I metodi pubblici fungono da comunicatori con il controller, che può sfruttarli per richiedere i dati digitati o selezionati, o può prenotarsi come ascoltatore in caso venissero premuti bottoni nell'interfaccia grafica.

Scelta del mazzo e del colore

La scelta del mazzo e del colore del tavolo di gioco è un'aggiunta rispetto alla versione CLI. L'implementazione prevede l'utilizzo di radio buttons che permettono all'utente la scelta, in particolare un gruppo di radio buttons per il mazzo e uno per il colore. La scelta del mazzo prevede anche un'anteprima di una carta per ogni set. Questo ha richiesto un particolare ridimensionamento del mazzo in versione "pixel", in quanto quel tipo di immagini ha dimensione e risoluzione diversa rispetto agli altri due mazzi.

Implementazione grafica del tavolo

La classe che ha richiesto più tempo e impegno per la realizzazione è stata `BlackjackSwingTableImpl`, in quanto prevede la creazione di più zone, la creazione di diverse schermate e bottoni per tutte le possibili situazioni durante una partita, e la gestione del

timer per l'assicurazione e le mosse.

La gestione di input e output interattivi durante la partita è gestita attraverso finestre che appaiono nei momenti dedicati. Ognuna di queste oscura la finestra principale sottostante impedendo al giocatore di proseguire senza prima aver confermato una scelta oppure confermato un esito.

Gestione dei timer

Il timer per l'assicurazione e per le mosse è gestito attraverso la classe Timer di Swing. Il countdown aggiorna una label ogni secondo in modo da rendere visibile in tempo reale il conto alla rovescia. Il timer si ferma se il giocatore prende una scelta o se invece scade e la scelta non è stata compiuta, gestendola quindi in maniera forzata.

La distribuzione delle carte avviene con un delay per ognuna, in questo modo il gioco risulta essere più fluido e anche la riproduzione del suono della consegna è più naturale. Questa implementazione avviene attraverso l'utilizzo di un ulteriore timer della stessa classe del precedente, che sfrutta un delay per eseguire tutte le operazioni inerenti alla consegna della carta, come la ricerca del file immagine o la riproduzione del suono. Il timer si ferma se le carte nella mano da mostrare sono terminate. La gestione della consegna delle prime quattro carte avviene attraverso un metodo apposito, le altre attraverso metodi che mostrano una carta alla volta.

Runnable

La sequenzialità delle operazioni o delle finestre è possibile attraverso il passaggio di Runnable ai metodi della view, che al verificarsi di precise condizioni, vengono eseguiti per far proseguire correttamente il flusso.

Gestione del doppio esito

Nel caso di partite con uno split, è stato pensato di aggiungere un doppio esito della partita nella stessa finestra. Questo ha richiesto una modifica al metodo che mostra l'esito finale, in modo che ricevesse come parametro non più un solo esito, ma una lista di esiti per entrambe le mani.

Frame principale

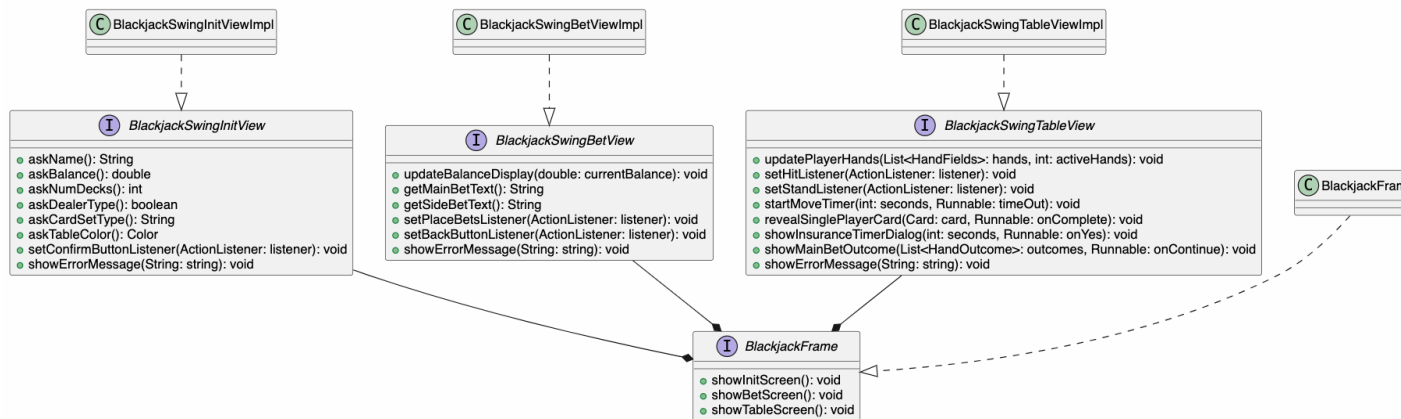
La classe della view che gestisce e mantiene al suo interno un'istanza per ognuna delle altre tre classi view è BlackjackFrame. Questa classe è quella contenuta nel controller ed è lei che, attraverso dei getter, mette a disposizione l'accesso alle altre classi della view più specifiche. Inoltre, è la classe che si occupa della creazione del frame principale.

Classi manager

Nel package view sono presenti anche due classi che fungono da manager di audio e carte: AudioManager e CardManager. AudioManager è una classe che gestisce tutti i suoni del gioco. Si occupa di recuperarli attraverso il loro percorso e di riprodurli quando i suoi metodi vengono chiamati dalle classi della view. La gestione del suono del menù principale avviene attraverso un loop. CardManager è una classe che viene utilizzata dalle view in versione Swing per recuperare le carte attraverso l'URL, in base ai dati ricevuti dal controller. Una volta recuperate e gestite in dimensione, queste vengono restituite e poi mostrate.

Gestione degli errori

In tutte le classi è presente un metodo che mostra un pannello con una segnalazione e uno specifico messaggio derivante dalle eccezioni lanciate nel model. Questo permette di rendere chiari gli errori e di mostrare correttamente al giocatore le operazioni illegali.



2.2.3 Controller nel dettaglio

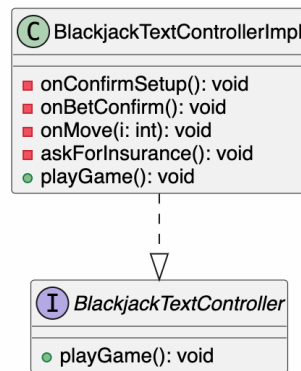
Come spiegato nelle sezioni precedenti il controller è stato diviso, lasciando che ogni versione di view (CLI e GUI) abbia la propria versione di controller.

Controller CLI

Il controller della versione CLI presenta un unico metodo pubblico che viene richiamato dalla classe main per far iniziare il flusso di gioco. Gli altri metodi sono privati e fungono da comunicatori con la view. Richiamano metodi della view, salvano i dati ottenuti e li comunicano al model, oppure passano parametri da mostrare.

Aggiornamento dei record

I record vengono aggiornati mano a mano che prosegue il flusso del gioco nel metodo pubblico. I dati che non cambiano nel corso della partita vengono mantenuti in variabili locali, mentre quelli soggetti a variazione (punteggio, carte, stato delle mani) vengono utilizzati per costruire una nuova istanza del Record ad ogni aggiornamento, così da fornire sempre alla view una fotografia coerente e aggiornata dello stato di gioco. Prima di essere passati ai metodi della view i record vengono riassegnati in questo modo, così da avere i dati sempre aggiornati.



Controller GUI

Il controller GUI non presenta alcun metodo pubblico per la gestione ad eventi dell'interfaccia che non prevede la necessità della gestione di un flusso di gioco continuo. Nel costruttore del controller in versione Swing, vengono chiamati i metodi della view per mettersi in ascolto di eventuali segnali dai bottoni dell'interfaccia, in modo da ricevere eventuali messaggi e muoversi di conseguenza. Proprio il controller svolge il ruolo di listener nel pattern scelto.

Anche il controller in questa versione presenta metodi che richiamano metodi della view per ottenere da questa dati e metodi che invece chiedono alla view di mostrare quelli ricevuti dal model.

Gestione mano del dealer

La gestione della mano del dealer, così come la mano del player, viene gestita sfruttando un delay con un timer. La logica delle carte e della mano è lasciata completamente al model, il controller riceve solo la lista di carte completa. Prima viene scoperta la carta inizialmente coperta, poi con un delay tra una carta e l'altra e un suono viene mostrata tutta la lista aggiornando il punteggio.

Gestione delle finestre

Le varie finestre vengono gestite attraverso un uso condizionale. Una finestra riguardante una particolare funzionalità, se non obbligata, viene mostrata solo se il giocatore potrebbe essere interessato. Ad esempio, la schermata di vincita della side bet viene mostrata solo se il giocatore ha puntato; la stessa cosa vale per la schermata dell'esito dell'assicurazione.

Gestione dell'audio

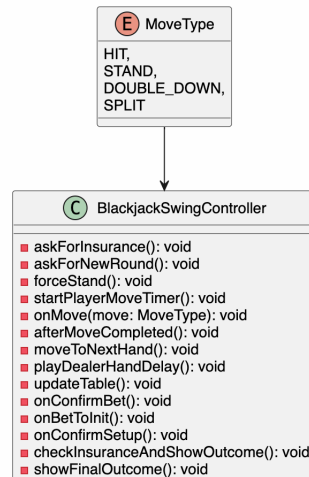
Proprio nel controller vengono richiamati i metodi definiti nella classe AudioManager. Ogni volta che viene eseguita un'operazione che richiede un suono, il controller richiama il metodo e gli passa il nome del file, che poi il metodo nel manager unirà al suo percorso.

Gestione mano attiva

Nel controller avviene anche la gestione della mano attiva in modo da tenere traccia della mano su cui effettuare le operazioni. In particolare, dopo ogni mossa eseguita, viene chiamato un metodo che controlla lo stato della mano ed eventualmente aggiorna la mano

attiva. All'inizio di un nuovo round viene azzerato il contatore.

In generale in entrambi i controller vengono raccolte le eccezioni lanciate dal model, in questo modo è possibile mostrare messaggi chiari all'utente invece che errori da terminale. Entrambe le versioni raccolgono le eccezioni e richiamano il metodo per mostrare gli errori della view, passandogli il messaggio proprio dell'eccezione lanciata dal model.



Capitolo 3

SVILUPPO

3.1 Testing automatizzato

Il testing automatico è stato eseguito per verificare il corretto funzionamento del model e il corretto cambiamento degli stati di gioco. Per ogni classe implementativa del model è stata affiancata una classe di testing automatizzato che sfrutta la libreria JUnit 5. Questi test sono stati pensati per verificare ogni caso limite, ogni combinazione particolare e il comportamento del sistema in determinate situazioni prestabilite. Le classi di testing sono suddivise per metodi; ognuno di questi esegue test su una parte specifica della classe. Sono stati pensati anche metodi che verifichino il corretto lancio delle eccezioni qualora si dovessero verificare situazioni non accettabili per parametri o stati. Il testing è stato affiancato alle classi nel momento in cui è iniziata l'implementazione di classi che ne contenevano altre, in modo di verificarne la corretta combinazione. Ad ogni test riportante esito negativo veniva sempre affiancata la correzione del codice corrispondente, per poi rieseguire il test. Le modifiche permanenti venivano salvate solo quando tutti i test scritti riportavano esito positivo, cercando quindi di aggiungere sempre codice corretto e testato. Ad implementazioni o correzioni successive, veniva sempre legata una serie di test per validarne la correttezza. Proprio per questo approccio è stato piuttosto semplice legare il model alle due differenti versioni di view e controller sapendo già del corretto comportamento degli oggetti in gioco. Le classi di testing implementate sono:

- CardImplTest
- ShoeImplTest
- HandImplTest
- PlayerImplTest
- DealerImplTest
- BlackjackTest

La parte legata all'interfaccia grafica o a linea di comando non è stata testata automaticamente. Il suo corretto funzionamento è stato verificato manualmente, cercando appositamente di generare errori o provando a compiere azioni illegali o anche poco intuitive. Questa pratica ha portato alla necessità di numerosissime correzioni soprattutto per la parte grafica, costringendo più volte alla revisione della view o del controller.

3.2 Metodologia di lavoro

3.2.1 Version Control

Come strumento per il controllo del codice è stato utilizzato Git. Svolgendo il progetto autonomamente non ho dovuto affrontare situazioni di unione di parti scritte da diversi membri, ma ho comunque cercato di mantenere il più possibile un flusso di implementazione pulito. Il codice principale e funzionante risiede nel branch "main". Le aggiunte che comportavano qualche implementazione o qualche refactor che poteva compromettere il codice già funzionante e testato, veniva scritto in un branch apposito con commit mirati e pubblicati solo se correttamente funzionanti. Una volta terminata l'implementazione, il branch veniva unito al main. Durante l'implementazione è stato necessario anche creare contemporaneamente più di un branch per implementare feature diverse e non legate tra loro. Un esempio è il branch per l'implementazione della GUI, che è stato a volte portato avanti assieme ad altri branch per altre implementazioni non collegate.

3.3 Note di sviluppo

3.3.1 Feature avanzate

- **Lambda:** uso di espressioni lambda per la gestione degli eventi Swing (ActionListener su bottoni e componenti dell'interfaccia) e per la definizione di Runnable.
- **Runnable:** uso di Runnable, passati come callback ai metodi della view per garantire la sequenzialità di operazioni asincrone (es. rivelazione delle carte con delay).

3.3.2 Algoritmi non forniti da libreria

Calcolo del punteggio con gestione dinamica degli assi

Il calcolo del punteggio di una mano richiede una gestione particolare degli assi, il cui valore può essere 1 oppure 11 a seconda di quale sia il valore del punteggio totale. L'algoritmo implementato conta il numero di assi presenti nella mano e calcola inizialmente il punteggio considerandoli tutti con valore 11; se il totale supera 21, il valore di un asso alla volta viene ricondotto a 1 (sottraendo 10 al totale), ripetendo l'operazione finché il punteggio non rientra nel limite o non sono stati riconvertiti tutti gli assi disponibili. Inoltre in seguito al calcolo viene effettuato un controllo. In caso di presenza di un asso con valore 11 viene assegnato valore true al campo che registra la presenza di un asso soft. Questa informazione è determinante per la logica del dealer, che deve eventualmente pescare un'ulteriore carta anche su un punteggio di 17 se la mano ha un asso soft e la regola specifica è attiva.

Calcolo dell'esito della side bet Perfect Pairs

La side bet Perfect Pairs viene valutata esclusivamente sulle due carte iniziali della mano del giocatore, prima che vengano effettuate altre pescate. L'algoritmo confronta il valore nominale delle due carte per verificarne l'uguaglianza; in caso positivo, distingue tre livelli di vincita in base al seme: coppia dello stesso seme (paga 25:1), coppia dello stesso colore ma seme diverso (paga 12:1), coppia di colore diverso (paga 6:1). Il metodo

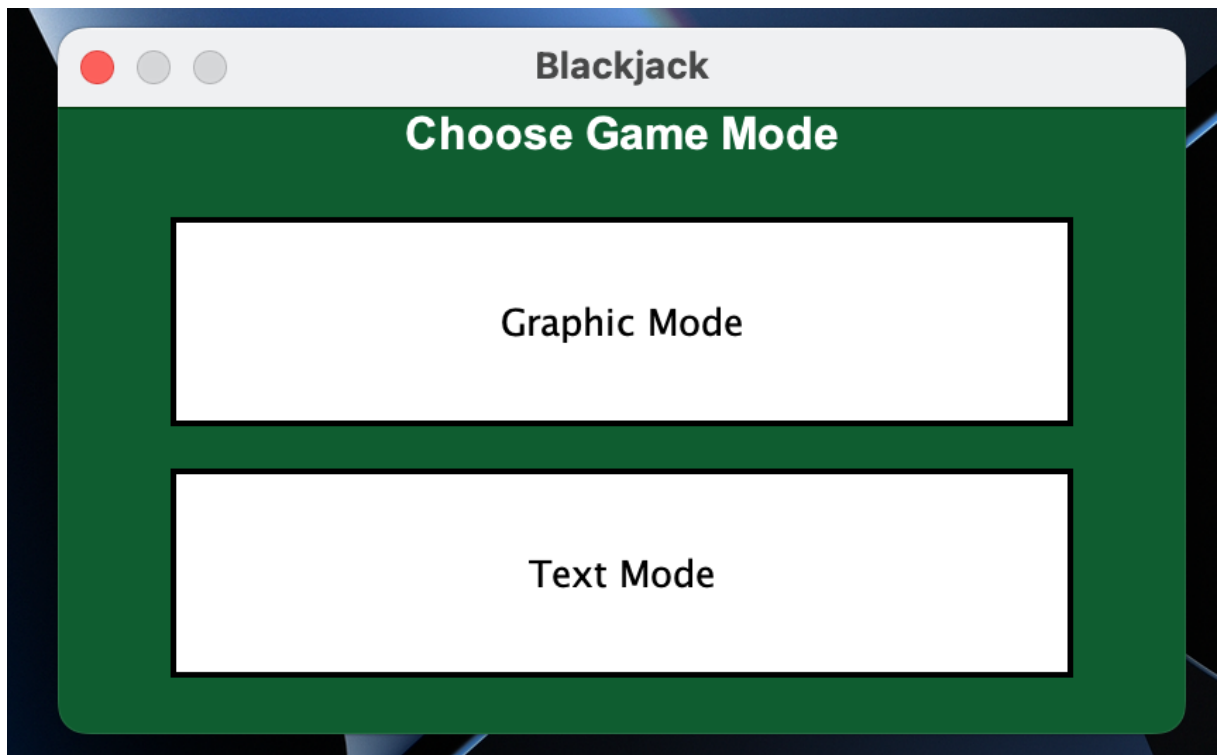
che effettua questo calcolo lancia un'eccezione se invocato quando la mano contiene più di due carte, poiché la side bet ha significato solo nella configurazione iniziale della mano.

Capitolo 4

TUTORIAL APP

In questa sezione, l'obiettivo è mostrare le principali parti e schermate dell'applicazione per renderle più chiare all'utente che si appresta a giocare.

4.1 Scelta della modalità



In questa schermata vediamo il pop-up di scelta iniziale per la modalità di gioco. I due bottoni sono collegati alle due modalità messe a disposizione. Una volta fatta la scelta, verrà aperta la schermata corrispondente.

4.2 Menù di inizializzazione

♣ ♦ BLACKJACK CASINÒ ♦ ♠

TABLE CONFIGURATION

Player name (max 30 characters):

Starting balance (max 1000€):

Number of decks (2-8):

Dealer Soft Hit (y/n):

☐ Activate the Soft 17 rule (Hit on soft 17)

Choose the set of cards:



PixelCards



ModernCards



ClassicCards

Choose the color of the table:

☒ Green

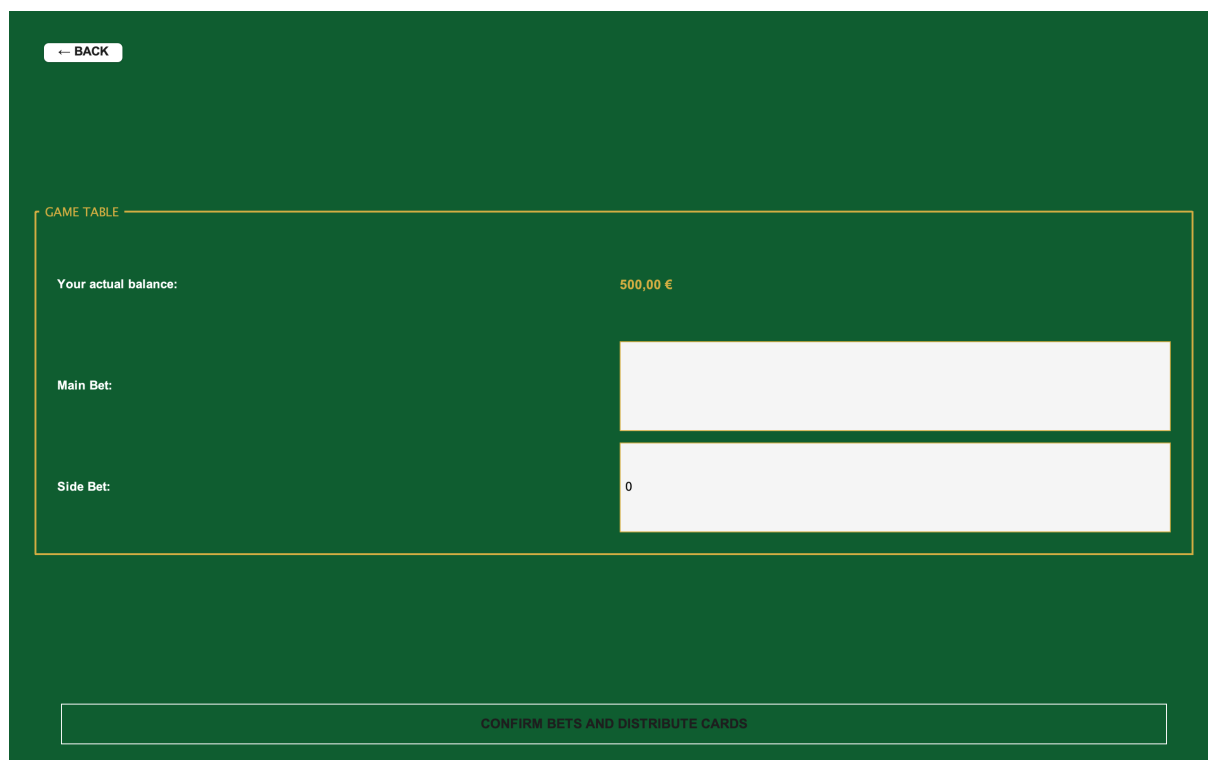
☐ Red

☐ Blue

ENTER GAME TABLE

Questa è la schermata principale. Qui vanno inseriti i principali dati riguardo l'utente e selezionate le impostazioni grafiche e regolamentari del gioco. La scelta grafica prevede tre mazzi diversi e tre diversi colori per il tavolo da gioco.

4.3 Menù per le bet

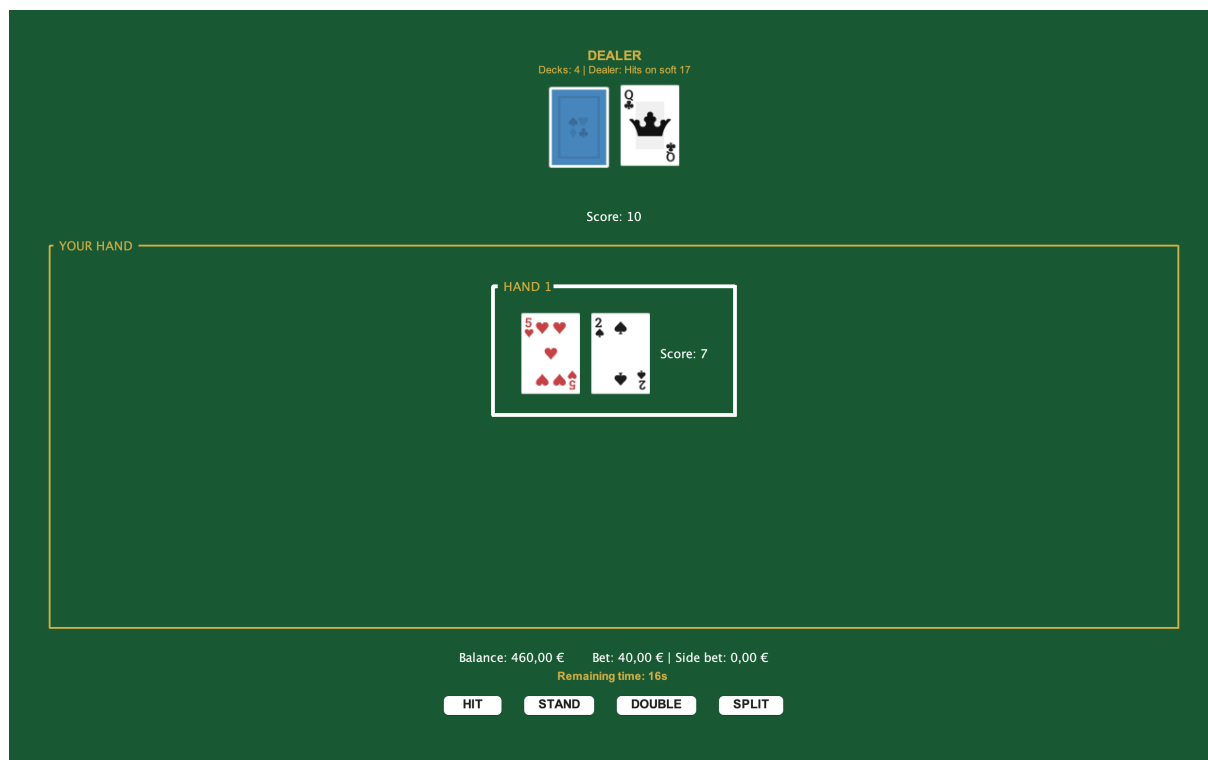


The screenshot shows a betting interface with a dark green background. At the top left, there is a white button with a left arrow and the text "BACK". Below this, a yellow-bordered box contains the following elements: the text "GAME TABLE" in yellow; "Your actual balance:" followed by "500,00 €" in yellow; "Main Bet:" followed by a large white input field; and "Side Bet:" followed by a white input field containing the number "0". At the bottom of the screen, there is a wide white button with the text "CONFIRM BETS AND DISTRIBUTE CARDS" in green.

Questa è la schermata per le bet. Qui è possibile inserire ogni round la bet principale e la side bet con cui entrare in partita. Il saldo del giocatore è presente per tenerlo alla mente dell'utente. Il bottone in alto a sinistra serve per tornare alla schermata iniziale e cambiare le impostazioni per iniziare da capo una nuova partita.

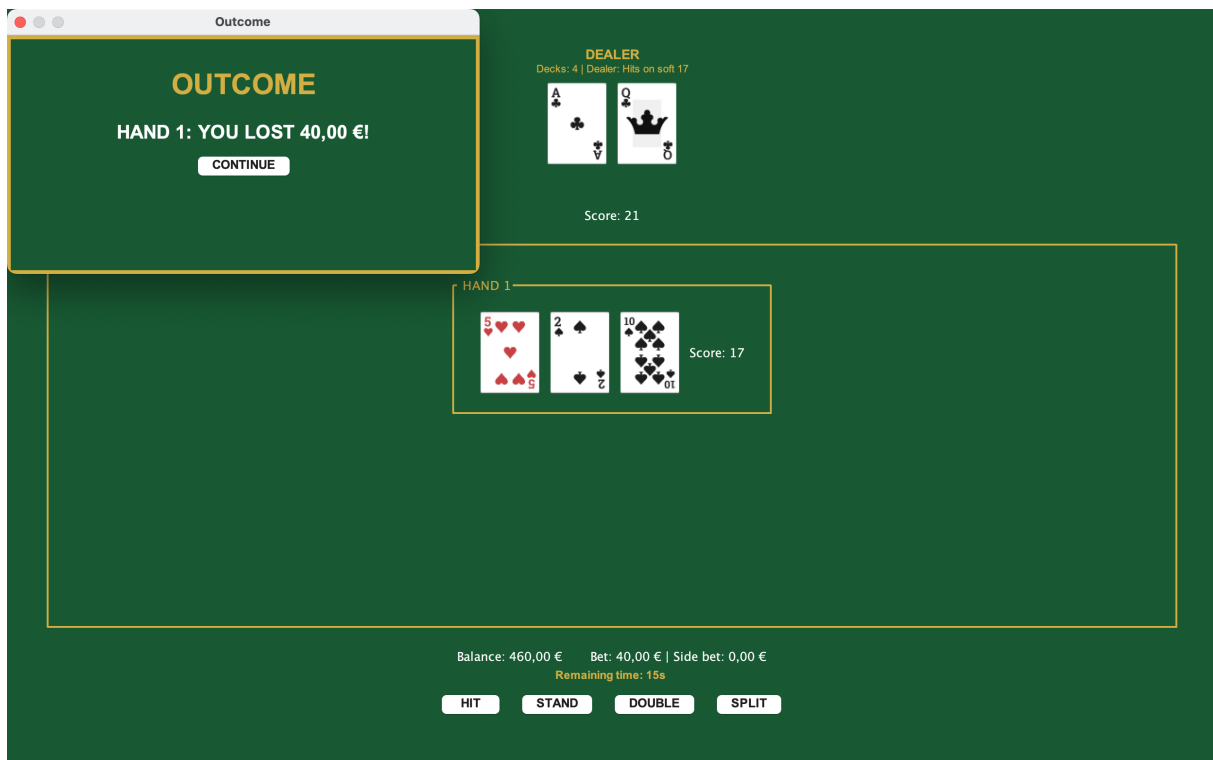
4.4 Tavolo da gioco

4.4.1 Tavolo iniziale



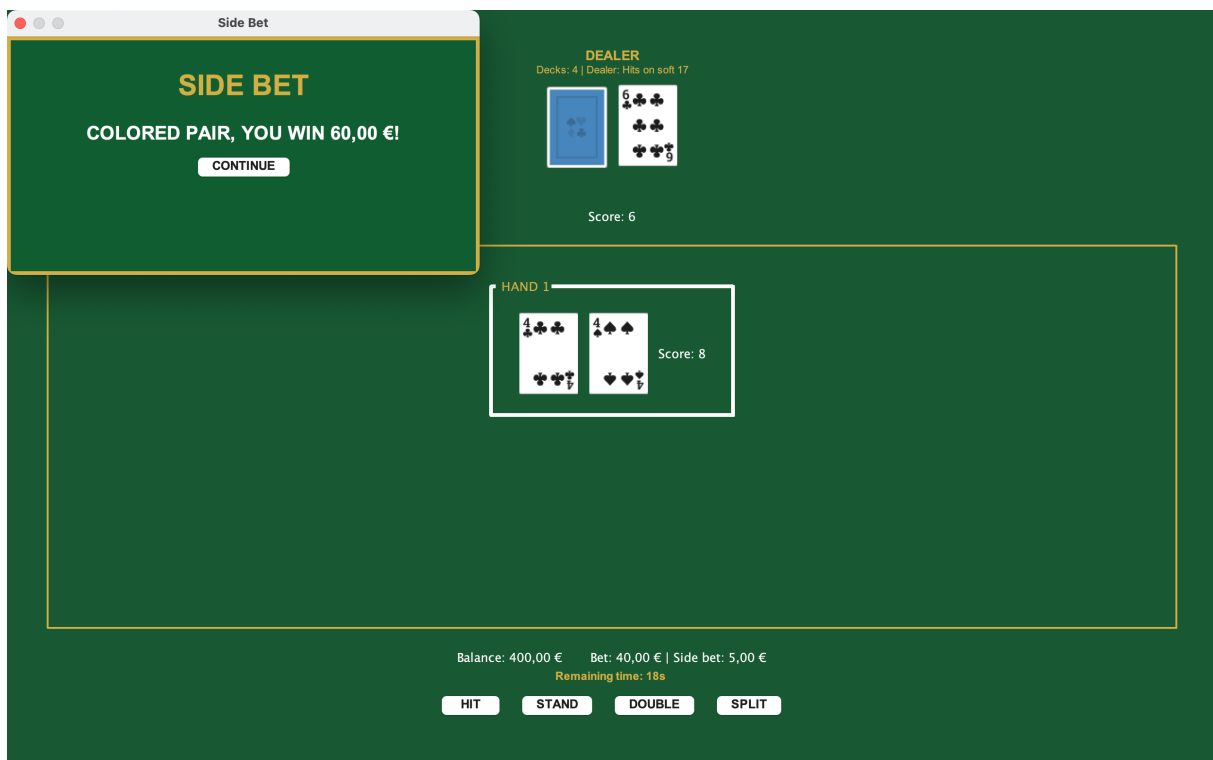
Questo è come si presenta il tavolo appena iniziato il round e consegnate le prime 4 carte. La zona per il player è quella a cui verranno aggiunte mano a mano le carte una volta eseguite le mosse. La carta del dealer è mostrata coperta così come il suo punteggio che è mostrato parzialmente. Nella parte bassa sono presenti i 4 pulsanti per le quattro possibili mosse. Qualora venisse eseguita una mossa non consentita, un pop up contenente il messaggio d'errore apparirebbe a schermo.

4.4.2 Mano sconfitta



Qui viene mostrato il pop up con l'esito finale della mano. Il giocatore perde la mano e quindi viene mostrato il messaggio di sconfitta. Seguirà il pop up con la richiesta per una nuova partita.

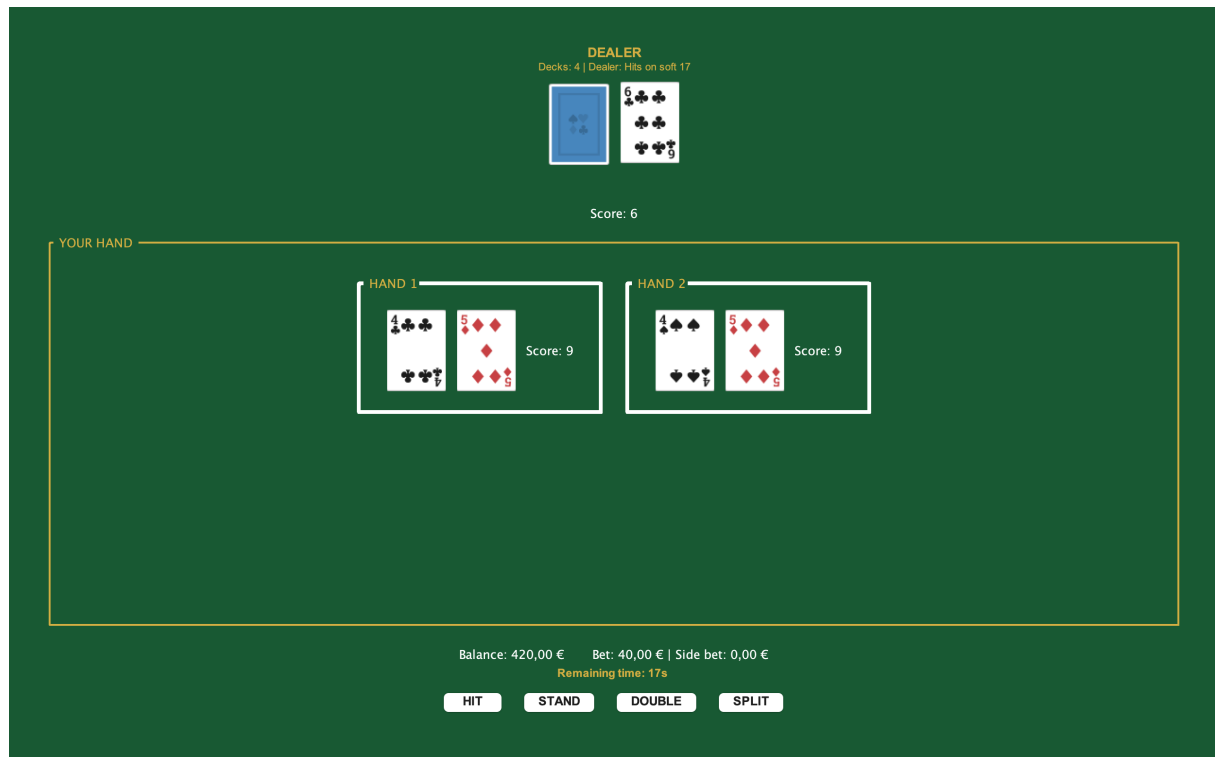
4.4.3 Esito side bet



All'inizio della mano, se il giocatore ha puntato sulla side bet, apparirà una finestra con

l'esito della bet. In questo caso, avendo una Coloured Pair, la scommessa risulta vinta e il giocatore ottiene i soldi.

4.4.4 Mano con split



Qui viene mostrato come cambia la configurazione del tavolo dopo uno split. Si crea una vera e propria nuova zona per una nuova mano e si prosegue con la partita. Il flusso è, prima la mano di sinistra, e poi la mano di destra.